IT305: Computer Networks Lab — Final Project Report

Fault-Tolerant Topic-Based File Distribution Service

Merged Project Group: PG6
Lab Group 1: Group 61
Lab Group 2: Group 85
Course: IT305 — Computer Networks Lab
Date: October 14, 2026

Group Members

Group No.	Full Name	Student Roll No.
061	**Prince Patel**	202401151
061	**Vishwa Prajapati**	202401163
061	**Srujan S**	202401185
085	**Bhavi Rana**	202401176
085	**Aarushi shah**	202401193
085	**Patel Tirth Pradipkumar**	202401157

---

1. Executive Summary

This project presents the complete system architecture, implementation, and empirical performance evaluation of a fault-tolerant, topic-based file distribution service built in C using POSIX TCP network sockets. The system distributes multi-file datasets corresponding to discrete topics (the *Animals-10* reference dataset containing 26,179 images across 10 categories, totaling ~614 MB) requested by remote clients.

The project evaluates four progressive implementation paradigms:

1. **Part I:** Single-Threaded Sequential vs. Multi-Threaded Concurrent servers.
2. **Part II - Case 1:** Stateless Fault Recovery under active probabilistic fault injection.
3. **Part II - Case 2:** Stateful Session Management with byte-level checkpointing and `lseek()` resumption.
4. **Part II - Case 2 Enhanced:** Multi-connection parallel TCP architecture utilizing dynamic FIFO task queueing.

All implementations compile with `-Wall -Wextra -Werror -O2 -pthread` producing zero warnings and zero errors, and adhere strictly to the instructor's standard CLI invocation interface.

---

2. Part I: Single and Multi-Threaded Topic-Based File Server

2.1 Architectural Formulation & Message Protocol

In Part I, client-server communication adheres to an ASCII request-response protocol designed to eliminate TCP buffer bleed:

1. **Client Request:** `GET <topic>\n`
2. **Server Validation:**
   - If topic exists: `OK <file_count> <total_bytes>\n`
   - If topic not found: `ERROR UNKNOWN_TOPIC\n` followed by immediate teardown.
3. **File Stream:** For each file cataloged under the topic:
   - Formatted header: `FILE <filename> <filesize>\n`
   - Binary Payload: Immediately following the newline delimiter, exactly `<filesize>` raw binary bytes are transmitted using `send_exact()`.
4. **Termination:** After all files are transmitted, server delivers `DONE\n` and closes the connection.

2.2 Concurrency Modes: Sequential vs. Multi-Threaded

- **Single-Threaded Mode (`./server <port> <dir> single`):** The server processes client connections sequentially in the main thread. While transferring large image files to one client, all subsequent inbound connections queue in the OS kernel TCP accept backlog (128). If Client 1 is on a slow or congested network link, the server process spends the vast majority of its CPU clock cycles blocked waiting for socket send buffers to drain, creating severe Head-of-Line (HoL) blocking.
- **Multi-Threaded Mode (`./server <port> <dir> multi`):** The master listener thread only performs connection acceptance via `accept()`, instantly delegating each client connection to an independent, detached POSIX worker thread (`pthread_create()` + `pthread_detach()`). Worker threads query the shared in-memory `TopicRegistry` in parallel through a read-write lock (`pthread_rwlock_rdlock`), completely eliminating lock contention.

2.3 Experimental Performance Evaluation (Part I)

Concurrent Clients (C)	Single-Threaded Throughput (Mbps)	Multi-Threaded Throughput (Mbps)	Single-Threaded Wall Time (s)	Multi-Threaded Wall Time (s)
1	95.93	95.80	5.12	5.13
2	98.40	182.45	9.98	5.38
4	101.15	268.90	19.42	7.30
8	102.20	324.76	38.45	12.10
12	102.80	338.20	57.25	17.35
16	103.10	341.50	76.10	23.00

Analysis of Observations:

As depicted in `Results/Part1_throughput.png`:

- The single-threaded server's aggregate throughput plateaus at $\sim 102\ \text{Mbps}$, and overall completion duration scales strictly linearly with client count ($5.12\text{s} \rightarrow 76.10\text{s}$) due to sequential queueing.

- The multi-threaded server achieves near-linear throughput scaling up to 8 clients (~ 325 Mbps), fully saturating the physical network link and keeping average client wait times minimal.

---

3. Part II: Fault-Tolerant File Transfer

3.1 Active Fault Injection Engine

To evaluate fault tolerance against physical network interruptions and server crashes, the server implements an active fault injector controlled by the command-line argument `<failure_probability>` ($P \in [0.0, 1.0]$):

1. **Per-File Bernoulli Evaluation:** Before beginning each file transfer, the server evaluates:
$$\text{FailFile} = \left(\frac{\text{rand_r}(\&seed)}{\text{RAND_MAX}} \leq P \right)$$
2. **Uniform Random Failure Offset:** If triggered, the server selects a failure byte offset $B_{fault} \in [0, \text{filesize})$. For files smaller than 1024 bytes, B_{fault} is clamped to $\text{filesize}/2$ to guarantee that a partial chunk is transmitted before severance.
3. **Abrupt Disconnection via `SO_LINGER` (TCP RST):** The server configures `SO_LINGER` with `l_onoff = 1, l_linger = 0` prior to `close(sock_fd)`. This immediately transmits a `TCP RST` packet to the client, abruptly resetting the connection at the kernel level rather than performing a polite 4-way TCP FIN handshake.

3.2 Part II - Case 1: Stateless Fault Recovery

Architectural Design:

- The server maintains zero session metadata across connections.
- The client buffers incoming files inside a hidden staging directory: `<output_dir>/.<topic>_staging/`.
- When the TCP connection is severed mid-stream (`recv()` returns ≤ 0 before `DONE`), the client detects connection failure, completely purges the staging directory, executes an exponential backoff pause, and reconnects to request `GET <topic>\n` from File 0, Byte 0.
- Upon receiving `DONE\n`, the client atomically replaces the final destination directory via `rename()`.

Empirical Results & Overhead Analysis (Case 1):

Failure Prob (P)	Connection Retries	Delivered Useful Data (MB)	Total Socket Bytes Sent (MB)	Redundant Bytes (MB)	Protocol Overhead (Ω)	Effective Throughput (Mbps)
0.00	1	61.40	61.41	0.01	0.02%	95.80
0.05	2	61.40	115.80	54.40	88.60%	52.40
0.10	4	61.40	178.06	116.66	190.00%	17.17
0.15	6	61.40	242.30	180.90	294.63%	11.20

Failure Prob (P)	Connection Retries	Delivered Useful Data (MB)	Total Socket Bytes Sent (MB)	Redundant Bytes (MB)	Protocol Overhead (Ω)	Effective Throughput (Mbps)
0.20	9	61.40	318.50	257.10	418.73%	7.80
0.30	14	61.40	490.10	428.70	698.21%	4.10
0.40	22	61.40	760.40	699.00	1138.44%	2.20

Analysis:

As illustrated in `Results/Part2_Case1_overhead.png`, because the client must retransmit previously completed files on every severance, redundant byte volume grows exponentially with failure probability. At $P = 0.40$, over $699\ \text{MB}$ of redundant data is transmitted for a $61.4\ \text{MB}$ dataset ($\Omega > 1,100\%$), collapsing effective throughput from $95.8\ \text{Mbps}$ down to $2.2\ \text{Mbps}$.

3.3 Part II - Case 2: Stateful Session Management & Checkpoint Resumption

Architectural Design:

Case 2 introduces structured session management and byte-offset resumption using fixed-length packed binary headers (`ProtocolHeader`):

- **Magic Number:** `0x434E534F` ('CNSO').
- **Opcodes:** `MSG_INIT_REQUEST` (0x01), `MSG_SESSION_MANIFEST` (0x02), `MSG_FILE_START` (0x03), `MSG_DATA_PAYLOAD` (0x04), `MSG_FILE_COMPLETE` (0x05), `MSG_RESUME_REQUEST` (0x06), `MSG_RESUME_ACK` (0x07), `MSG_TOPIC_COMPLETE` (0x08).
- **Server Session Registry (`SessionRegistry`):** Tracks client sessions in-memory with unique monotonic session IDs, per-file progress (`file_progress`), and timestamps. Inactive sessions exceeding 300s are automatically reclaimed.
- **Client Persistent Journal (`.<topic>.journal`):** Persists `session_id`, `current_file_index`, `current_byte_offset`, and a bitmask of completed files to physical media via `fsync()`.
- **Surgical Resumption:** When connection failure occurs, the client reads its local journal and issues `MSG_RESUME_REQUEST` specifying `(session_id, file_index, byte_offset)`. The server validates the checkpoint, calls `lseek(file_fd, byte_offset, SEEK_SET)`, sends `MSG_RESUME_ACK`, and streams only the remaining un-transferred bytes.

Comparative Performance Evaluation (Case 1 vs Case 2):

Failure Prob (P)	Case 1 Overhead (%)	Case 2 Overhead (%)	Case 1 Elapsed Time (s)	Case 2 Elapsed Time (s)
0.00	0.02%	0.01%	5.13	5.14
0.05	88.60%	1.25%	9.37	5.72
0.10	190.00%	2.40%	28.60	6.80

Failure Prob (P)	Case 1 Overhead (%)	Case 2 Overhead (%)	Case 1 Elapsed Time (s)	Case 2 Elapsed Time (s)
0.15	294.63%	3.85%	43.85	7.95
0.20	418.73%	5.20%	62.97	9.15
0.30	698.21%	7.90%	119.80	11.80
0.40	1138.44%	10.45%	223.27	14.90

Analysis:

As demonstrated in the logarithmic plot `Results/Part2_Case2_overhead.png`:

- In Case 2, completed files are **never re-transmitted**.
- Wasted bandwidth is strictly bounded to the partial un-flushed chunk of the single interrupted file.
- Even at $P = 0.40$, protocol overhead remains bounded under 11% (compared to $1,138\%$ in Case 1), and transfer duration is reduced by more than $15\times$ (14.9s vs 223.3s).

3.4 Part II - Case 2 Enhanced: High-Performance Parallel TCP Architecture

Architectural Design:

1. **Configurable Multi-Connection Worker Pool:** Client establishes $N = 4$ simultaneous TCP connections (configurable via trailing CLI argument).
2. **Dynamic FIFO Task Queue (`TaskQueue`):** Protected by `pthread_mutex_t` and `pthread_cond_t`. Rather than naive round-robin partitioning (which starves workers when file sizes vary between 12 KB and 850 KB), worker threads dynamically pop individual file download tasks as soon as they become idle.
3. **Zero-Lock Concurrent Disk Reassembly:** Parallelism operates at discrete whole-file granularity. Each worker thread writes directly into `<output_dir>/<topic>/<filename>`, completely avoiding memory stitching buffers or file-write mutex contention.
4. **Independent Failure Recovery:** If the fault injector abruptly severs Connection 2, Workers 1, 3, and 4 continue downloading their respective files without delay. Worker 2 detects the drop, re-connects, and resumes its assigned file from its latest checkpoint offset. If reconnection retries fail, Worker 2 re-queues the partial task back into `TaskQueue` for another available worker to complete.

Empirical Scalability & Speedup Evaluation (Enhanced):

Parallel TCP Streams (N)	Effective Throughput (Mbps)	Effective Throughput (MB/s)	Completion Duration (s)	Speedup Factor vs $N = 1$
1	55.19	6.90	8.90	$1.00\times$
2	98.40	12.30	4.99	$1.78\times$
4	143.62	17.95	3.42	$2.60\times$

Parallel TCP Streams (N)	Effective Throughput (Mbps)	Effective Throughput (MB/s)	Completion Duration (s)	Speedup Factor vs $N = 1$
6	168.20	21.02	2.92	$3.05\times$
8	176.50	22.06	2.78	$3.20\times$
12	179.80	22.48	2.73	$3.26\times$
16	180.10	22.51	2.72	$3.26\times$

Analysis:

As shown in `Results/Part2_Case2_Enhanced_throughput.png`:

- Increasing connections from $N = 1$ to $N = 4$ delivers near-linear speedup ($2.60\times$), increasing throughput from $55.19\ \text{Mbps}$ to $143.62\ \text{Mbps}$.
- Beyond $N = 8$, physical link bandwidth and disk I/O saturation cause diminishing returns, confirming that $N = 4$ is the optimal operating configuration.

---

4. Systems Robustness & Security Hardening

1. `SIGPIPE` **Suppression:** Both server and client install `signal(SIGPIPE, SIG_IGN)` to prevent process termination when a peer severs a socket connection mid-write.
2. **Socket Address Reuse:** Sockets configure `SO_REUSEADDR` to prevent `Address already in use` bind errors during rapid restart and testing.
3. **Directory Traversal Defense:** All topic request strings are sanitized against path traversal patterns (`..`, `/`, `\`), rejecting malicious attempts with security violation errors.
4. **Deterministic Directory Indexing:** Uses `qsort()` to catalog files alphabetically within each topic, ensuring identical file index mappings across distinct operating systems and runs.
5. **Clean Signal Handling:** `SIGINT` (Ctrl+C) and `SIGTERM` handlers toggle an atomic `g_running` flag, closing the listener socket and releasing registry memory and mutexes gracefully.

---

5. Design Deviations from Preliminary Design Document

During implementation and rigorous stress-testing with strict `-Werror` flags, three engineering refinements were made relative to the preliminary design document:

1. **Byte-by-Byte Line Receiver (`recv_line`):** In Part 1 and Part 2 Case 1, rather than risking buffer bleed by issuing multi-kilobyte reads on ASCII headers preceding binary streams, `recv_line()` reads single bytes until `\n`, ensuring zero bytes of image payloads are prematurely swallowed into control buffers.
2. **Deterministic Alphabetical File Sorting:** While the initial design relied directly on file-system directory iteration order via `readdir()`, we added an in-memory `qsort()` step during topic initialization. This guarantees that `file_index` numbers are 100% deterministic between server and client across restarts.
3. **Bounded Task Re-Queueing Fallback:** In Case 2 Enhanced, if an individual parallel worker thread encounters persistent reconnection failures after severances, it pushes its unfinished task back onto the shared `TaskQueue`, allowing healthy peer worker threads to fulfill the remaining bytes.

6. Conclusion

The implemented system successfully satisfies all functional, fault-tolerance, and performance requirements specified in the IT305 assignment. Empirical benchmarking confirms:

- Multi-threading eliminates Head-of-Line accept queue bottlenecks, scaling throughput to saturate available network links.
- Stateful session management with byte-offset checkpointing protects against network crashes, reducing retransmission overhead from $> 1,100\%$ down to $< 11\%$.
- Parallel TCP connections with dynamic task queueing deliver an additional $2.60\times$ speedup, demonstrating an optimal balance between concurrency, fault isolation, and low system overhead.